BetaQuest -- A Rock Climbing Route Finder

Summer Alwood

4/20/2026

Yilmaz - Period 3

Table of Contents

Abstract

Rock climbing presents a uniquely complex pathfinding challenge because progress depends on coordinated limb movement, balance, reach, and constantly shifting body geometry. To explore how these factors can be modeled computationally, we developed a route-finding system that generates human-like climbing sequences on indoor bouldering problems. Climbs are represented as graphs derived from manually collected hold-quality assessments and image-based measurements of hold size and position. A recursive search algorithm evaluates potential moves using a weighted quality function that incorporates hold strength, distance, biomechanical penalties, and simple movement heuristics. The system selects moves by examining short chains of future actions and choosing the path with the highest cumulative quality. Testing on real climbs showed that the algorithm reliably produced sequences that reached the designated top hold, but the resulting movement often felt rigid, occasionally overestimating reach or suggesting unstable limb configurations. These outcomes highlight both the promise and the limitations of simplified, limb-based modeling. This work establishes a foundation for more advanced approaches that incorporate center-of-gravity estimation, dynamic movement patterns, richer hold geometry, and more nuanced representations of human climbing behavior.

I. Introduction, Background, and Applications

Rock climbing, as a fairly constrained task, has the potential to be a good medium of exploring pathfinding for specifically human movement. Although, at its core, the objective is simple -- simply reach the top of a wall using provided holds -- it quickly becomes complex when considering how it requires the complete utilization of the human body's ability to move. Considerations of center of gravity, reach, weight distribution, balance, chaining moves, as well as the sheer number of possible moves from a given position on any climb, drastically increase the complexity -- and decrease the chance -- of a perfectly implemented rock climbing "routefinder." To ensure results, although imperfect, would be achieved within its duration, this particular implementation constrained the task to completely flat walls (using a 2-D grid) in order to eliminate additional gravitational complexity that comes with overhung routes as well as to only single moves (as chains introduce too many possibilities which exponentially decrease computational efficiency). In further research, it would be essential to address these factors (see section VI).

Beyond its technical interest, this project also has the potential to benefit the rock climbing community. Existing solutions such as Kaya and Mountain Project rely on human-generated route information and are largely limited to popular, established climbs, making them far less useful for indoor gyms where routes are reset frequently. Additionally, very few existing pathfinding algorithms focus on human movement at the level of individual limbs; current approaches in robotics often involve humanoids mimicking movements extracted from video, while "human-like pathfinding" in video games typically treats the body as a single object rather than a coordinated system of limbs. A tool that can simulate realistic climbing movement could therefore serve as a valuable resource, particularly for new climbers, by helping them understand how climbing feels and reducing the initial challenge of determining how to move up the wall.

The language used throughout this paper will adhere to the following definitions:

1. Define a **position** as a record of the locations of all four limbs.
2. Define a **move** as a change of position
3. Define a **route** as a sequence of moves [that begins with the **start position**, with both feet on the floor and both hands on the designated start hold(s), and ends with the **top position**, defined by having both hands on the designated "top" hold]

II. Methods:

Please note that the information here represents the final result of the project. For information on trial and error and more reasoning as to why certain choices were made, please see Sections C (General) and D (Quality Function) of the Appendix.

Overview

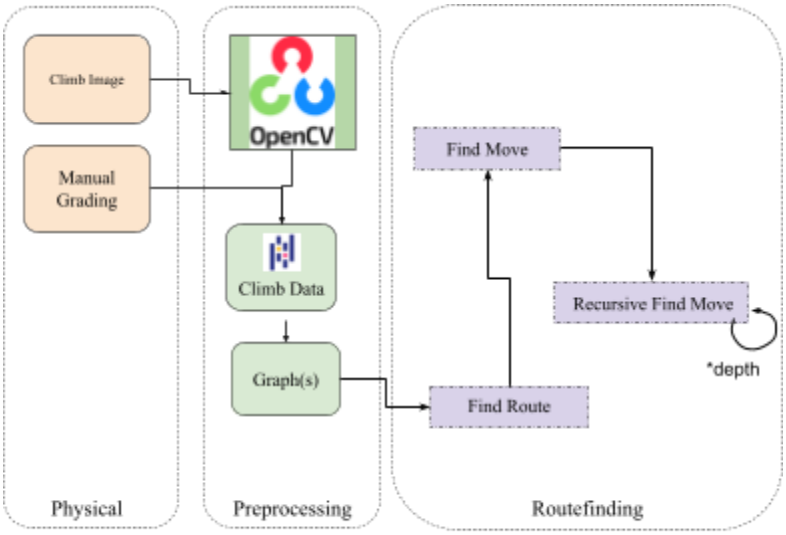


Figure 1: System Architecture Diagram

In the broadest sense, the goal of this algorithm is to convert information about a climb to a sequence of moves going from the start position to the top position. To represent the climb digitally, we use a graph where each node represents a hold and neighbors being defined as any other hold with a position within a certain radius. The basic algorithm to find a route is as follows: iterating beginning at the start position and looping until they've reached the top position, for each opportunity to move (i.e. position before the loop ends), for each possible move from that position, find every chain of a certain length beginning with that move. Finally, choose the move with the highest quality chain attached to it. See Detailed Route-Finding Algorithm for a more detailed description. The overview of the method used from start (the climb itself) to finish (the final output route) is shown in *Figure 1*.

The main method used is fairly simplistic as compared to the well-established standards of route-finding methods such as A*, Dijkstra's, or Breadth/Depth First Search[5]. This is for a few reasons. The first is that these more robust algorithms are typically optimized for

single-agent systems, which are not appropriate for this application. Even algorithms created specifically for multi-agent systems[3][4] are focused on non-interference of agents, rather than optimizing their relative positions. Thus, none of these were applicable to this particular scenario.

The code used in this project was written entirely in Python. The materials and packages used in this project include:

1. Onshape, PLA Filament, and a 3-D printer to print a calibration square for data taking.
2. OpenCV[9] for image processing.
3. Pandas[10] for data management
4. Matplotlib[11] for graph visualization.

Data Collection

Because the aim of this research is to explore the algorithmic side of creating a route finder for rock climbing, it was necessary to simplify the gateway process of representing a physical climb digitally. The data taken for this purpose was done at the Alexandria location of Sportrock Climbing Centers on 3 different days: 10/2/2025, 12/26/2025, and 12/31/2025. A total of 14 climbs were recorded. See Appendix F for statistics involving the data.

Data Structure & Attributes

To represent climbs, the data used in this project took each hold as an instance, recording 12 different attributes for each. These are as follows:

1. (MANUAL) Route Code -- An 8 character representation of the route to which each given hold belongs. It follows the format "[wall code]-[color code]-[grade]". The grade follows the traditional bouldering grades for climbs, choosing the lowest grade associated with the color tag assigned to the climb by the routesetter. Specifics about the wall and color codes can be found in Appendix E.
2. (MANUAL) Whether or not the hold is a start -- A boolean value that is TRUE if the routesetter assigned that hold as [one of] the start hold[s].
3. (MANUAL) Whether or not the hold is the top hold -- A boolean value that is TRUE if the routesetter assigned that hold as the final hold.

4. (MANUAL) Estimated Strength Score -- A float from 1-10 that is the average of several climber's evaluation of the hold's quality. *Replaced by UDLR Strength Scores in graph.*
5. (MANUAL) Up, Down, Left, Right Strength Scores -- Floats from 1-10 that are estimates of the hold's quality when being held on each side. One of these will be the same as the Estimated Strength Score (usually this is the strongest direction, as it is the one climbers would choose to use).
6. (IMAGE) X and Y coordinates -- Floats representing the x and y position of the hold's center relative to the defined origin of a local coordinate system. These are found during the image processing phase.
7. (IMAGE) Size_x and Size_y -- Floats representing (half of) the width and height, respectively, of the hold. These are found during the image processing phase.

Manual Data

The manual data-taking process consists of two phases. The first phase takes place in the climbing gym where, for each route, we first take a picture of the wall with our chosen climb, placing our 0.2m x 0.2m calibration square in the bottom right. Then, at least 3 people climb the route [optionally, they are recorded to establish potential human-generated betas] and estimate the quality of each hold, which is recorded. This quality estimation is done blind, meaning that no climber may hear another's rating, so that their own rating is not influenced. After all three climbers rate each hold, the estimated strength score is calculated as the average of their individual ratings. The second phase consists of estimating the different region strengths using the estimated strength score (i.e. the strength of the best region) and the image of the climb. This estimation is done by one climber (for time efficiency) soon after leaving the gym.

Image Data & Preprocessing

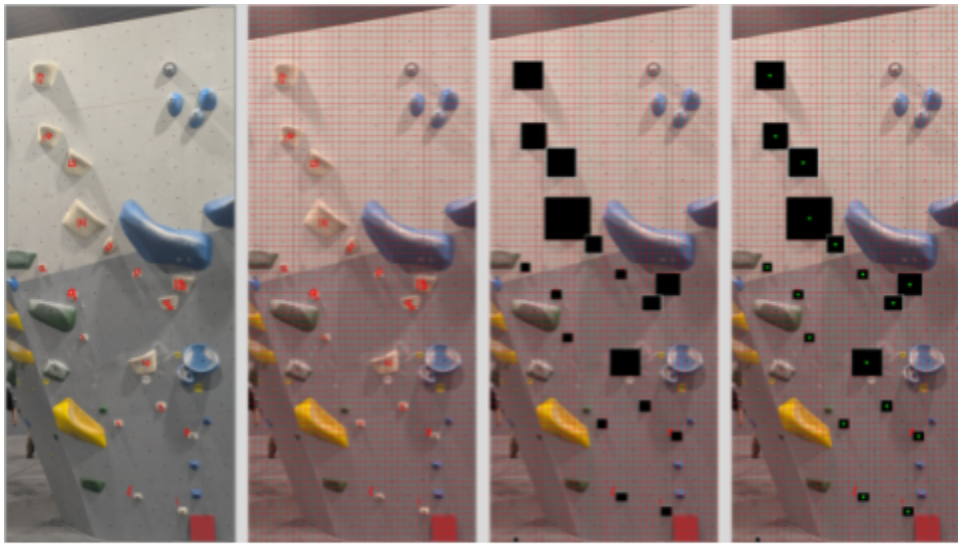


Figure 2: Image Processing Phases

The whole process of analyzing a climb image to extract information is shown in *Figure 2*. The picture taken at the beginning of manual data processing (phase 1) was also used to measure the size and position of each hold. Using OpenCV,[9] the image was parsed in order to find the red calibration square in the bottom right. The pixel dimensions of the square are divided by its known physical dimensions (0.2m x 0.2m) in order to get a pixel per meter (PPM) conversion both vertically and horizontally for the entire image. That measure was then used to overlay a 0.1m grid over the image. Next, because of issues with OpenCV detecting holds on its own (See Appendix C), it was necessary to manually identify holds and overlay boxes with their approximate dimensions for the code to detect. Then, the images with overlaid boxes were passed through another OpenCV script which extracts the size and coordinates of each block (in pixels). The information about each "blob" (hold) from a given climb was stored under a data structure identified by its route code.

Using the route code, we then associated the manually taken and image data of a climb. The conversion of size and coordinates is converted from pixels to meters at this step as well. Finally, the data was combined into a single dataframe containing every climb and every attribute, and saved under a new csv file to be used in routefinding methods.

Route-finding (Detailed)

Data Structures & Representation of Physical Characteristics

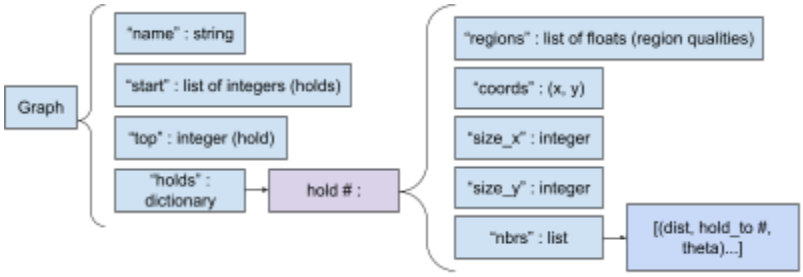


Figure 3: Graph Structure

Climbs are represented as a dictionary that can be read as a graph, the structure of which is visualized in *Figure 3*. Position is represented as a tuple of length 4, with each entry being a length 2 tuple of form (Hold #, Region). Each entry represents a different limb, in the order: left hand, right hand, left foot, right foot. For example: ((8, "Up"), (9, "Right"), (2, "Up"), (0, "Up")), which translates to "Left hand on top of hold 8 / Right hand on the right of hold 9 / Left foot on top of hold 2 / Right foot on ground." Furthermore, limbs have numerical values associated with them which correspond to their index in the position tuple. The left hand corresponds to index 0, while the right foot corresponds to index 3.

Moves are represented by a tuple of length 3 of the format (Limb moving, Limb's current position (hold), Limb's prospective position (hold)). For example: (1, 6, 10), which translates to "Right hand from hold 6 to hold 10."

Core & Iterative Methods

Finding the Best Region of a Hold

Theta (θ) is calculated as the angle formed by the horizontal and the line drawn between the centers of two neighboring holds ($\theta = tan^{-1}(\Delta y/\Delta x)$). The base priority list, based on the value of θ, is as follows (the base version of this defines flatness_threshold as $\pi/3$, or 60 degrees). If θ is positive, that implies that the initial hold is to the left of the destination hold.

Condition	Priority List	Reasoning

$0 < \theta <$ flatness_threshold	[Up, Right, Left, Down]	The climber will usually consider holding the top of the hold first, but a more horizontal angle means they should attempt the sides before the bottom. If θ is positive, the climber should prioritize the right over the left.
-flatness_threshold $< \theta < 0$	[Up, Left, Right, Down]	The climber will usually consider holding the top of the hold first, but a more horizontal angle means they should attempt the sides before the bottom. If θ is negative, the climber should prioritize the left over the right.
flatness_threshold $< \theta$	[Up, Down, Right, Left]	If θ is mostly vertical, the climber should prioritize the vertical regions. If θ is positive, the climber should prioritize the right over the left.
$\theta <$ -flatness_threshold	[Up, Down, Left, Right]	If θ is mostly vertical, the climber should prioritize the vertical regions. If θ is negative, the climber should prioritize the left over the right.

A degree of randomness was also incorporated, adding a 5% chance of swapping two elements of the priority list. It is most likely (2.5% chance) to swap the first two elements and there is an equal chance (1.25% chance each) to swap either the second and third or third and fourth elements. Multiple swaps may happen.

After determining the priority by which we are considering regions, we iterate through the regions in priority order and choose the first one that meets the strength threshold (in this case, 6). The strength of this region is used in quality calculations as the strength of the hold.

Evaluating Move Quality

In order to evaluate the quality of a given move, three groups of metrics were considered: 1) the destination hold's strength, based on the evaluation by climbers, 2) the distance between the two holds, and 3) various manually added rewards and penalties based on more complex ideas such as balance, limb extension, and matching. All three were given weights (although, in the final version, they all were set to 0.7).

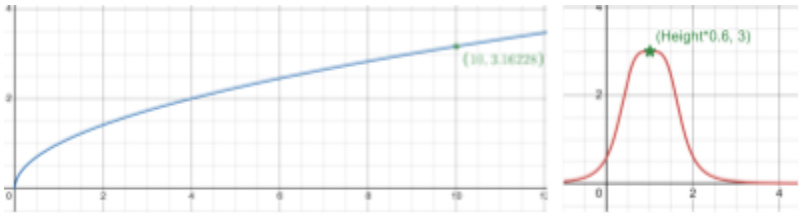


Figure 4: Strength (Left) and Distance (Right) Transformations

Strength and distance were transformed based on nonlinear functions; for strength, the square root of the raw value was used, as we hypothesized that the practical difference between

higher quality values was less than that of lower quality values. For distance, the function $\frac{0.75}{(x-height*0.6)^4+0.25}$ was used because of its symmetry, its horizontal asymptote on the x axis, and its relatively flat top (which is more forgiving of values close to the chosen "optimal" distance than, for instance, a simple parabola), see visualization in *Figure 4*. Both of these functions have very close maximum values (3.16 and 3.0), which allow their measurements to be easily comparable when choosing their relative weights.

Type	Description	Condition or Calculation	Weight
Reward	Taking a foot off the floor	if hold_from == 0: value = 1	20
Reward	Putting a hand on the top	if hold_to == graph["top"]: value = 1	5
Penalty	Moving a limb downwards	if to_y < from_y: value = 1	-10
Penalty	Putting two limbs on the same hold	if hold_to in position: value = 1 (10*holdto[sizex]+1)	-2
Penalty	Left limb is further right than its right counterpart	#for both hands and feet… if left_x > right_x: value = left_x - right_x	-6
Penalty	Hands (or feet) are at too different heights	#for both hands and feet… if left_limb[y] - right_limb[y] > 0.5: value = 1	-0.5
Penalty	Imbalance: if average x position of hand is too different from that of feet	value = (avg_hand_x - avg_foot_x)^2	-5

Penalty	Extension: how stretched out various limb pairs are	opposite_hf_threshold = height*1.15
		same_hf_threshold = height*1.1
		hand_threshold = h*0.85
		feet_threshold = 0.7
		
		#for each limb pair…
		extensions.append(dist(limb1, limb2)/threshold)
		
		value = sum(10.67*((a-0.5)^6) for a in extensions)
			-3

Figure 5: Implemented Rewards and Penalties

The third element of quality, consisting of various rewards and penalties, was much less straightforward and harder to estimate a theoretical maximum or minimum value for. However, these quality modifications are the means by which we sought to address many of the more complex elements of human motion which could not be discerned simply from hold quality or distance. Several of these were related to the relative positioning of limbs. Each was given an individual weight (which represents their significance relative to one another) and [most] have base values ranging from 0 to 1. The specific elements are detailed in *Figure 5*.

Finding the Best Possible Moves

```
moves = []

for every limb L at hold H1:
        for every hold H2 that neighbors H1:
                move M = from H1 to H2
                find the best region R for M
                calculate the distance from H1 to H2

                calculate the quality of M

                if quality meets the threshold:
                        add (quality, distance, strength, M) to moves

sort moves
return M for best 5 moves
```

Figure 6: find_moves Pseudocode

In all routefinding methods, it is necessary to narrow down the many possible moves from any given position to a few optimal ones based on their quality and possibility. This is the purpose of the find_moves method; it loops through the possible moves for each limb and calculates its quality. Then, it compares every possible move (using distance, then strength as a tiebreaker) and returns the 5 best ones to be used in routefinding. The pseudocode of the algorithm used for this step is found in *Figure 6*.

Iterative Routefinding

```
position = feet on floor, hands on start(s)
route = [position]

while not both hands on top:
        next move = find best move
        add next move to route
        position = make_move (position, next move)

return route
```

Figure 7: find_route Pseudocode

The quickest routefinding method employed is the iterative method, which selects the move of highest quality at each position until the climber has reached the top position. Although this provides a feasible route, it lacks the sophistication of the final recursive methods and is more likely to move into a “dead end” position from which there are no feasible moves because it lacks information about future positions. The pseudocode of the algorithm used for this step is found in *Figure 7*.

Recursive Routefinding

```
given a position
moves = get best 5 moves
qualities = []

for move, quality in moves:
        pos = make move (move, pos)
        get path_q from recursive method →
        qualities.append((quality + path_q, move))

return move with highest quality path
```

```
if depth is 0, return 0
moves = get 5 best moves
qualities = [-100]; boost = 0

for move, quality in moves:
        pos = make move (pos, move)
        path_q from this method w/ depth - 1

        if topping out: boost = 20; depth = 1

qualities.append(quality + path_q)

return max(qualities)
```

Figure 8: Recursive Method Pseudocode

The final, more optimal route-finding method used in this project applies the same logic as the previously described iterative route-finding method, but incorporates recursion to apply foresight. Rather than simply considering the single next move, it looks at every chain of n (using n=5) moves beginning with each move it considers, and chooses the move that begins the highest quality chain. This prevents the algorithm from going down a suboptimal path that seems strong but weakens one or two moves deep. Additionally, in order to this to function properly with the previously established quality calculation, two quality modifications were done on this level: 1) adding 5 * the remaining # of moves to the quality if the chain contains topping out and 2) adding 20 to the quality of the chain if the first move is taking one's foot off the floor. The former was meant to encourage routes that reached the top in fewer moves and the latter was added because the recursion otherwise encouraged the climber to move their hands before both feet were off the ground, which is an invalid action. The pseudocode of the algorithm used for this step is found in *Figure 8.*

III. Results:

Data was taken on three different climbs, processed, and run through the recursive route finder using depth = 5. Default weight calibration (decided by sight reading on previously taken climbs) was used, which was likely suboptimal.

Climb 1

Figure 9: Algorithm Output for FL-BK-V1

Of the three climbs tested in this round, the algorithm performed the worst on climb FL-BK-V1, the results for which are shown in *Figure 9*. You may note that the climber fails footwork at move 9 because the move suggested was not physically possible or very difficult to execute. Additionally, at the very end of the climb, the algorithm asked that the climber move their hands too far from their feet, resulting in the climber needing to move his feet in between moves 17 and 18 in order to feel stable. When giving feedback after attempting the algorithm-generated route, the climber stated that, although some parts of the route felt like a decent beginner beta, there were times when hand placement was backwards, reach was too far, and the suggested sequence made the climb feel a grade harder than it actually was.

Some of the issues with this particular route may have been because of the feature (large black section) in the middle of the climb that contains holds 11 and 13 alongside the slight slant of the wall on the left side, both of which cause the climb to fail our assumption of a perfectly flat wall. Although this failed assumption may have contributed to the issues with reach on the last few moves, it was clear that the algorithm was overestimating the climber's reach.

Climb 2

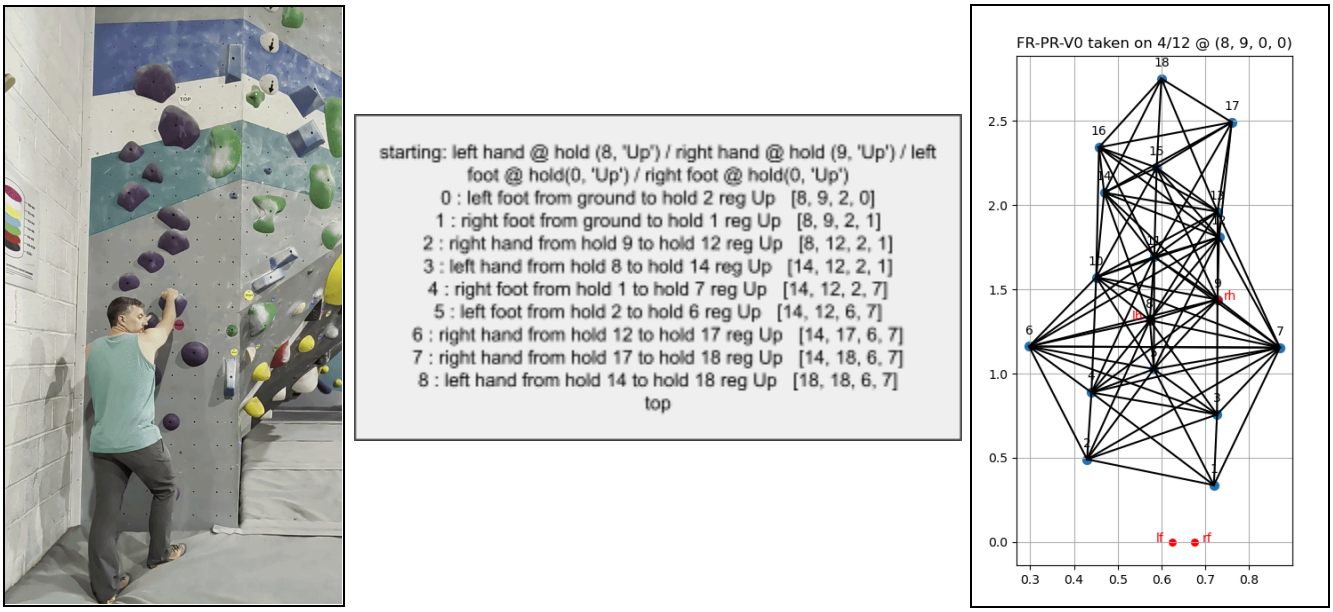


Figure 10: Algorithm Output for FR-PR-V0

For the three climbs tested in this round, the algorithm performed the best on climb FR-PR-V0, the results for which are shown in *Figure 10*. A grade easier than the worst-performing climb, this route was much more interconnected and passed the assumption of having a completely flat wall. The climber was able to complete the route nearly exactly as output (although he needed to execute moves 3, 4, and 5 in quick succession because the left hand reach was slightly too far; note that he briefly flags on the wall to execute the move before stepping up). When providing feedback, the climber said that this was a strong beginner beta for this climb; it felt efficient, balanced, and was a reasonable means of executing the climb.

IV. Conclusion:

Although the final product is able to provide a route, it remains imperfect; several moves are unnatural, backwards, or unnecessary. This research could be extended in many ways, including: accounting for non-flat walls, utilization of reinforcement learning with human feedback (RLHF) to optimize quality calculation, and allowing for move chaining, flagging, or accounting for hip movement. Additionally, it is extremely unlikely that a perfectly generalizable solution to this problem will be found; there is simply too much variance between climbs. It may benefit future researchers to adjust evaluation methods based on qualities of a climb as a whole.

Additionally, though the route is output both as text and visual, it was difficult for the climber to remember the route while on the wall. This limited the algorithm's effectiveness, as it led to movement being stilted and unnatural.

V. Limitations & Future Work:

This research aimed to explore the extremely complex issue of simulating and suggesting means of human movement in the context of rock climbing. Because the issue's scope was so large and it would be impossible to completely address in the time frame available, we were forced to drastically simplify it at nearly every step. Thus, there remains vast potential for improvement of research in this area.

Climb and Position Representation

The first stage in which improvements could be made is the representation of the climb itself. First, the means by which the climb is analyzed to create the graph could be much more accurate; the box shape approximation of each hold works well for some shapes, but is highly inaccurate for others. Future work should attempt to better represent the shape of each hold, especially as it pertains to regions and directionality of holds. Furthermore, when building the graph, it could be beneficial to build different edges for arms and legs to account for differences in reach (as opposed to using a circle of constant radius) between the different kinds of limbs. Other basic improvements could include accounting for features and the third dimension (expanding application to overhangs).

Ideally, the data collection process would be completely digital rather than manual, potentially scanning the wall in order to estimate the quality of holds. This would drastically widen the potential real-world application of this algorithmic solution because its usage would not require a long, specific and arduous data collection process with at least one experienced climber.

Furthermore, in its final version, the algorithm created in this project represents position as a tuple of length 4, with each entry representing a limb's position being a length 2 tuple of form (Hold #, Region). However, this means of representation makes it extremely difficult to estimate the climber's center of gravity because there are a range of possible positions their body could be in with the same limb position. To address this, we suggest adding a fifth element to the

positional argument representing the climber's hips or center of gravity, which could be further used to calculate weight distribution on each limb (which was a blind point we noticed during testing; the algorithm would attempt to move a limb that was supporting most of the climber's weight, which feels unnatural and is very difficult).

Quality Calculations

The quality calculations used in this project have monumental potential for improvement as, although they were able to capture some complexity, they were somewhat simple. First, the weighting of different values against one another (strength vs. distance vs. rewards and rewards/penalties vs. one another) was minimally explored and should be experimented with to find more optimal values. Additionally, more complex quality calculations that are not based on manually chosen equations could be attempted; for instance, using a neural network and training on multiple climber betas, though we would caution future researchers attempting this method as it requires a lot of data and may lead to overfitting to certain styles of beta. using reinforcement learning algorithms. Additionally, consideration of the climb itself/its characteristics would likely benefit future algorithms, as those vary widely (grade, types of holds, connectedness) between climbs and will drastically affect the optimal route.

VI. References:

[1] G. Kulathunga, A Reinforcement Learning based Path Planning Approach in 3D Environment. arXiv, 2021. Accessed: Apr. 14, 2026. [Online]. Available: https://arxiv.org/pdf/2105.10342

[2] H. Xie et al., “Language‐Conditioned Path Planning,” Proceedings of Machine Learning Research, vol. 229, pp. 1–15, 2023. Accessed: Apr. 14, 2026. [Online]. Available: https://proceedings.mlr.press/v229/xie23b/xie23b.pdf

[3] V. Desbois, O. Sankur, and F. Schwarzentruber, “An Efficient Modular Algorithm for Connected Multi‐Agent Path Finding,” ECAI 2024 – 27th European Conference on Artificial Intelligence, Santiago de Compostela, Spain, 2024. Accessed: Apr. 14, 2026. [Online]. Available: https://hal.science/hal-04650916/document

[4] I. Calviac, O. Sankur, and F. Schwarzentruber, “Improved Complexity Results and an Efficient Solution for Connected Multi‐Agent Path Finding,” AAMAS 2023 – 22nd International Conference on Autonomous Agents and Multiagent Systems, London, U.K., 2023. Accessed: Apr. 14, 2026. [Online]. Available: https://hal.science/hal-04075393/document

[5] U. Hybesis, “Pathfinding Algorithms: The Four Pillars,” Medium, 2023. Accessed: Apr. 14, 2026. [Online]. Available: https://medium.com/@urna.hybesis/pathfinding-algorithms-the-four-pillars-1ebad85d4c6b

[6] Kaya Climb, “Sportrock Alexandria Gym Information.” Accessed: Apr. 14, 2026. [Online]. Available: https://kaya-app.kayaclimb.com/gym/sportrockalexandria

[7] Mountain Project, “Rock Climbing Routes, Photos, and Guides.” Accessed: Apr. 14, 2026. [Online]. Available: https://www.mountainproject.com/

[8] Rockwerx Climbing, “Climbing Wall Design and Construction.” Accessed: Apr. 14, 2026. [Online]. Available: https://rockwerxclimbing.com/

[9] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, vol. 25, no. 11, pp. 120–123, 2000.

[10] W. McKinney, “Data Structures for Statistical Computing in Python,” Proceedings of the 9th Python in Science Conference, S. van der Walt and J. Millman, Eds., pp. 56–61, 2010. doi: 10.25080/Majora‐92bf1922‐00a

[11] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007. doi: 10.1109/MCSE.2007.55

[12] A. Li, S. Koenig, and T. Uras, “MAPF-LNS2: Fast Repairing of Multi-Agent Path Finding Solutions Using Large Neighborhood Search,” Computers & Graphics, vol. 99, pp. 1–12, 2021. Accessed:

Apr. 14, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0097849321001849

[13] J. Li, “Multi-Agent Path Finding (MAPF) Research Overview.” Accessed: Apr. 14, 2026. [Online]. Available: https://jiaoyangli.me/research/mapf/

[14] Y. Chen et al., “Neural Multi-Agent Pathfinding with Diffusion Models,” arXiv, 2026. Accessed: Apr. 14, 2026. [Online]. Available: https://arxiv.org/html/2601.07701v1

[15] A. V. T. Nguyen, M. H. Pham, and T. H. Tran, “A Review of Path Planning and Navigation for Autonomous Robots,” Frontiers in Neurorobotics, vol. 13, 2019. Accessed: Apr. 14, 2026. [Online]. Available: https://www.frontiersin.org/journals/neurorobotics/articles/10.3389/fnbot.2019.00036/full

Additional Links:

[a] Code: https://github.com/s-alwood/rockclimbing-routefinder

[b] Demo & Video of Presentation: https://docs.google.com/presentation/d/1KP34jTQwcAG_Eap3udZho7yWNlWMECV8PtPq82_ryxI/edit?usp=sharing

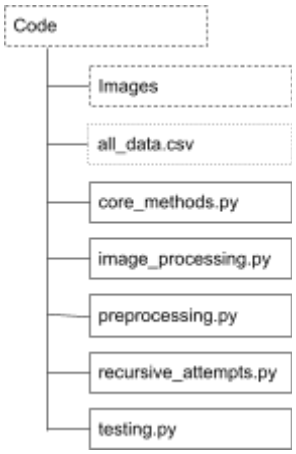
VII. APPENDIX:

A. DECLARATION OF AI USAGE:

ChatGPT was used to generate the OpenCV code which parsed images of climbs to find holds, the origin, and the red square for scale. Size conversions as well as all other methods were not generated by AI.

B. CODE:

Access code through open Github repository:
https://github.com/s-alwood/rockclimbing-routefinder
The repository also contains the STL file for the calibration square.



See structure of essential files (right). The repository includes several non-necessary but research-relevant folders and files such as problem_children and qualitydata.csv that may be interesting to browse but are not necessary for base functionality.

C. TRIAL AND ERROR LOG:

a) DATA TAKING AND PROCESSING

Data Attributes *Note: Each hold is an instance, associated into climbs using route code*			
Attempt #	Method Description	Motivation	Issues
Attempt 1	1. route code 2. alpha (α) -- estimated strongest angle a. may have multiple values 3. gamma (γ) -- ½ the width of the "acceptable range" of positions for the climber to hold 4. x & y coordinates (local reference frame) 5. estimated strength score	Using α and γ allow for a continuous angle measurement.	Difficulty estimating alpha and gamma introduced significant complexity into the data taking process. Potentially too complex to program as well (would require tracking the angle the climber's limb makes with the hold, for which there are far too many possibilities), with not

	(0-1)		enough perceived benefit.
Attempt 2	1. route code 2. if hold is the start 3. alpha (α) -- estimated strongest angle 4. may have multiple values 5. directionality (1-20, a low score meaning only strong from a small range of angles) a. used to estimate gamma 6. x & y coordinates (local reference frame) 7. size_x and size_y measures (width and height) 8. estimated strength score (1-10)	*see previous* The size of the hold impacts how good it can be, especially when matching limbs. Size could also be used to tweak distance (as opposed to only using the coordinates of the center)	The same issues with alpha and gamma. Although directionality made estimating gamma somewhat easier, it was still too complex to implement and introduced too much potential for inaccurate estimation.
Attempt 3 / Final	1. route code 2. if hold is the start 3. if hold is the top 4. estimated strength score (1-10) 5. estimated directional strengths (up, down, left, right quadrants) 6. x & y coordinates (local reference frame) 7. size_x and size_y measures (width and height)	Simplifying method while maintaining some measure of directional strength.	Strength is still an estimate. Quadrants are a clumsy approximation; don't hold up perfectly for holds that are tilted to one side. Future attempts should explore more regions, tilted quadrants, or revisit the α/γ representation

Image Scaling			
Attempt #	Method Description	Motivation	Issues
Attempt 1	Manual measurement of distances between holds with a tape measure	Simplest means, pairs with manual processing method and doesn't require a program	Too many measurements, excessively time consuming

Attempt 2	Using a feature of the wall itself (i.e. color patterns on the wall, a certain hold)	Established frame of reference so that measurement can be done in post-processing, thereby reducing time spent in the gym.	Requires manual image processing (excessively time consuming) as it is not consistent between images and therefore difficult to program
Attempt 3 / Final	Using a 3D printed 0.2m x 0.2m square placed in the bottom right of the images.	Established frame of reference that will be consistent between climbs.	Tilting of the square led to slight inflation of scale. Future work, if using a similar method, may prefer a circle with a consistent radius as rotation would not impact OpenCV's perception of its shape.

Image Processing			
Attempt #	Method Description	Motivation	Issues
Attempt 1	Use OpenCV to find the red calibration square and overlay a grid. Then, manually estimate the size and positions of each hold.	Simplest means, requires no complex programmatic image processing.	Too time consuming (takes approximately 1 hour per climb), too much approximation.
Attempt 2	Use OpenCV to find the red calibration square and overlay a grid. Then, use OpenCV to locate each hold and determine its size and position (converting pixels to meters using the determined scale).	Minimum human involvement, so should theoretically be the most precise.	Because the colors of the holds were not consistent (due to chalk, lighting, and other factors), OpenCV frequently missed or misidentified holds. The coordinate system used is unintuitive, as a low y-coordinate corresponds with a higher position.
Attempt 3 / Final	Use OpenCV to find the red calibration square and overlay a grid. Then, manually overlay black squares over each hold	Balance manual and programmatic processing. Minimize human involvement while still completing	Squares are a clumsy approximation of hold size and shape. Also increased time consumption of this process due to necessity of

	and place an origin somewhere in the bottom left. Finally, use OpenCV to locate the boxes and determine their sizes and positions (converting pixels to meters using the determined scale).	the task accurately.	manual processing.

b) ROUTEFINDING

see Appendix D for quality function modifications

Routefinding Methods		
Attempt #	Method Description	Issues
Attempt 1	Iterative #1 -- Until we reach the top, pick the best move and update position	Worked, but was shortsighted. Would dead-end occasionally.
Attempt 2 / Final	Recursive #1 -- Consider not only the move we would be making, but also the best (of every) chain of moves of a certain length (5) beginning with that move. Pick the move with the best chain.	More time consuming. Awkwardness with quality calculations (fixed). Optimal chain length not experimented with.

D. QUALITY FUNCTION - UPDATE LOG:

Date	Version	Changes
1/25/2026	v0	add weights = [0.7, 0.5] add distance = 0.5/((distance-0.5)**2+0.25) add strength = (10*strength)**(1/2) add base = s_w*strength + d_w*distance add (+1) topping out

		add (+10) coming off ground add (-2) going down add (-1) matching add impossible: feet too far, feet over hands
2/4/2026	v1	change distance = 0.5/((distance-0.5)**4+0.25) change strength = (strength)**(1/2) add (-2/(10*size_x + 1)) matching add (-dx) right going left or right going left
2/6/2026	v2	change weights = [0.7, 0.5, 0.8] *added reward/penalty weight* change distance = 0.75/((distance-0.75)**6+0.25)
2/19/26	v3	remove right going left or left going right add left limb more right than respective right limb (-dx)
2/24/26	v4	add unbalanced measure (-(1.5*abs(avgx_hands - avgx_feet))^2) add secondary quality modification to find_route → add fatigue calculation (quality -= 0.1*number of moves made with the associated limb)
3/3/26	v5	change feet strength bump 2 → 1.5 add extension evaluated for each indiv add penalty if feet/hands too far from one another change quality must > 0 change hand-hand pair impossible thresh to 0.75 change foot-foot pair impossible thresh to 0.7

		add weight for benefits & penalties 0.7 → 0.8
3/10/26	v6	add importance values within ben & pen (for easier weighting of indiv elements)
3/17/26	v7	change weights → [0.7, 0.7, 0.7] change center point of distance inversion to 0.25*height → 0.6*height change opposite hand-foot pair impossible thresh to 1.15 change same hand-foot impossible thresh to 1.1 add route_aware methods; quality bumped significantly if first move takes foot off ground

E. ROUTE AND COLOR CODES:

Wall codes:

- Entrance wall = EW
- Slab = SL
- Island right (on image) = IR
- Island left (on image) = IL
- Bayside = BY
- 10/20 = TT
- Topout (probably not used but if so) = TO

Color codes:

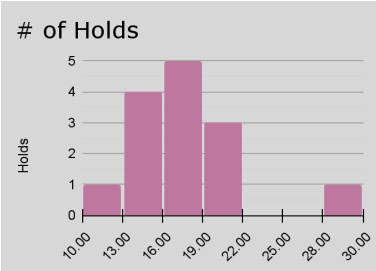
- Pink = PI
- White = WH
- Yellow = YL
- Blue = BL
- Green = GR
- Red = RD

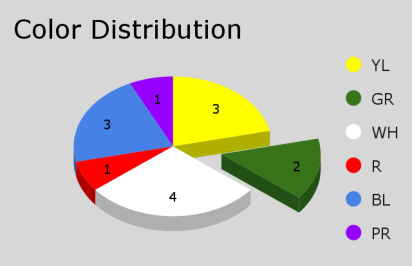
Sportrock Alexandria Floor Plan

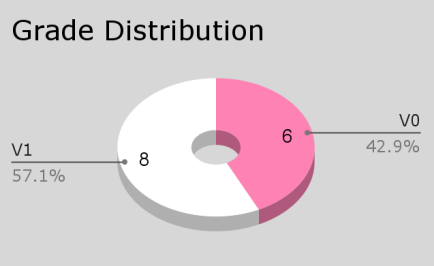
- Black = BK
- Orange = OR
- Purple = PR

F. CLIMB DATA STATISTICS:

Climb Information:







# of Climbs Recorded	14
# of Instances	244

Climber Information:

	Gender	Age	Height	Weight	Level	Dates Present
A	F	43	5'4	160	V0-V2, V3-V4	12/26/25, 12/31/25
B	M	44	5'9	200	V3-V4, V5-V6	10/2/25, 12/26/25, 12/31/25
C	F	15	5'3	135	V3-V4, V5-V6	10/2/25, 12/26/25
D	M	17	5'6	135	V0-V2	10/2/25
F	M	44	5'11	185	V3-V4, V5-V6	12/31/25

Date	# Climbs	Data Takers
10/2/25	5	C, B, D
12/26/25	3	C, B, A
12/31/25	6	A, B, F